



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC1253 - MATEMÁTICAS DISCRETAS

Tarea 6

19 de junio de 2025

1º semestre 2025 - Profesores P. Bahamondes - D. Bustamante - P. Barceló

Requisitos

- La tarea es individual. Los casos de copia serán sancionados con la reprobación del curso con nota 1,1.
- **Entrega:** Hasta las 23:59 del viernes 27 de junio a través del buzón habilitado en el sitio del curso (Canvas).
 - Esta tarea debe ser hecha completamente en $\text{\LaTeX}$. Tareas hechas a mano o en otro procesador de texto **no serán corregidas**.
 - Debe usar el template $\text{\LaTeX}$ publicado en la página del curso.
 - Cada solución de cada problema debe comenzar en una nueva hoja. ***Hint:*** Utilice `\newpage`
 - Los archivos que debe entregar son el archivo PDF correspondiente a su solución, junto con un **zip**, conteniendo el archivo **tex** que compila su tarea. Si su código hace referencia a otros archivos, debe incluirlos también.
- El no cumplimiento de alguna de las reglas se penalizará con un descuento de 0.5 en la nota final (acumulables).
- No se aceptarán tareas atrasadas (salvo que utilice su cupón **#problemaexcepcional**).
- Si tiene alguna duda, el foro de Github (issues) es el lugar oficial para realizarla.

Pregunta 1

Sea $G = (V, E)$ un grafo simple que no contiene un circuito hamiltoniano (es decir, un ciclo que pasa por cada vértice exactamente una vez). Comience a agregar arcos arbitrariamente a E hasta llegar a un grafo $H = (V, E')$ que contiene un circuito hamiltoniano (tal H debe existir porque un grafo completo siempre contiene un circuito hamiltoniano). Sea (a, b) el último arco que agregamos en este proceso. Por lo tanto, si sacamos el arco (a, b) de E' obtenemos un grafo $H' = (V, E' \setminus \{(a, b)\})$ que no contiene ningún circuito hamiltoniano.

Considere un circuito hamiltoniano en H . Tal circuito debe pasar por (a, b) . Podemos asumir entonces, sin pérdida de generalidad, que tal circuito hamiltoniano es de la forma:

$$(v_1 = a) \leftrightarrow (v_2 = b) \leftrightarrow v_3 \leftrightarrow \dots \leftrightarrow v_n \leftrightarrow (v_1 = a),$$

donde $\leftrightarrow$ denota que vértices consecutivos son adyacentes.

1. Demuestre que si b es adyacente a algún v_i en H' , para $3 \leq i \leq n$, entonces a no puede ser adyacente a v_{i-1} en H' .
2. En base a lo anterior, demuestre que $\delta_{H'}(a) + \delta_{H'}(b) < n$, donde $\delta_{H'}$ denota el grado de un nodo en H' .
3. Concluya que si $\delta_G(u) + \delta_G(v) \geq n$ para todo par u, v de vértices no adyacentes en G con $u \neq v$, entonces G tiene circuito hamiltoniano (donde, como antes, δ_G denota el grado de un vértice en G).

Solución

1. Suponga, por contradicción, que $b \leftrightarrow v_i$ y $a \leftrightarrow v_{i-1}$. Entonces $a \leftrightarrow v_{i-1} \leftrightarrow v_{i-2} \leftrightarrow \dots \leftrightarrow b \leftrightarrow v_i \leftrightarrow v_{i+1} \leftrightarrow \dots \leftrightarrow v_n \leftrightarrow a$ es un circuito hamiltoniano en H' . Esto es una contradicción.
2. El grado de b en H' es $i \leq n - 2$. Por punto (a), el máximo grado de a es entonces $n - 2 - i$. La suma de ambos grados es a lo más $n - 2$.
3. Asuma que no lo tiene. Entonces en el proceso explicado al construir H debemos en algún momento agregar un último arco (a, b) que hace que el grafo pase a tener circuito hamiltoniano. Pero $\delta_G(a) + \delta_G(b) \geq n$ implica $\delta_{H'}(a) + \delta_{H'}(b) \geq n$, y por tanto H' también tiene circuito hamiltoniano. Esto es una contradicción.

Pregunta 2

Cuando modelamos con lógica, convertimos una instancia de un problema y en una fórmula, de modo que la instancia es una solución del problema si y sólo si la fórmula es satisfactible. A esto se le llama **reducción**. En la práctica, requerimos reducciones que puedan realizarse en tiempo polinomiales con respecto al tamaño de la instancia del problema original.

Consideremos el siguiente algoritmo, que permite reducir el problema de k -coloración de grafos (es decir, decidir si un grafo puede colorearse con k colores sin que vértices adyacentes compartan color) al problema de satisfactibilidad booleana (SAT).

Algorithm 1: Reducción de k -Coloración a SAT

```
1 Function GraphKColoringToSAT( $G = (V, E)$ ,  $k$ ):
2    $\varphi \leftarrow \text{true}$ ;
3   foreach  $v \in V$  do
4      $C \leftarrow \text{false}$ ;
5     for  $i = 1$  to  $k$  do
6        $C \leftarrow C \vee c_{v,i}$ ;
7      $\varphi \leftarrow \varphi \wedge C$ 
8     for  $i = 1$  to  $k - 1$  do
9       for  $j = i + 1$  to  $k$  do
10         $\varphi \leftarrow \varphi \wedge (\neg c_{v,i} \vee \neg c_{v,j})$ ;
11  foreach  $(u, v) \in E$  do
12    for  $i = 1$  to  $k$  do
13       $\varphi \leftarrow \varphi \wedge (\neg c_{u,i} \vee \neg c_{v,i})$ ;
14  return  $\varphi$ 
```

1. Explique brevemente por qué la fórmula construida modela la k -coloración, es decir, que la fórmula modela que (a) a cada nodo se le asigna un único color y (b) dos nodos adyacentes no pueden tener el mismo color.
2. Para efectos del tiempo del algoritmo, consideraremos como operación representativa la asignación ($\leftarrow$). Demuestre que el algoritmo puede ejecutarse en tiempo polinomial.

Suponga que tiene un algoritmo **SAT-Solver** que, dado como input una fórmula en lógica proposicional, puede resolver el problema de satisfactibilidad booleana en tiempo $\mathcal{O}(f(n))$ donde n es el tamaño de la fórmula.

3. Construya un algoritmo que utilice a **SAT-Solver** y a **GraphKColoringToSAT** como subrutinas para determinar si un grafo es k -coloreable.
4. Estudie la complejidad de su algoritmo con notación $\mathcal{O}$.

Solución

1. El algoritmo básicamente está generando la fórmula vista en clases para 3-coloreabilidad de mapas, extendida a k colores. Cada variable $c_{v,i}$ representa que el nodo $v \in V$ está siendo coloreado por el color $i \in \{1, \dots, k\}$. Por lo tanto, el loop de las líneas 5 – 6 construye la disyunción que nos dice que el nodo correspondiente tiene algún color, mientras que el loop de las líneas 8 – 10 nos asegura que este loop es único (no puede tener a la vez dos colores), con lo que se cumple (a). Por otra parte, el loop de las líneas 11 – 13 toma los nodos vecinos y asegura que no pueden tener el mismo color, con lo que se cumple (b), es decir, la fórmula modela los dos puntos de la k -coloración.
2. La asignación de la línea 2 es llamada una única vez, la de la línea 4 es llamada $|V|$ veces, la de la línea 6 es llamada $|V| \cdot k$ veces, la de la línea 7 es llamada $|V|$ veces también, la de la línea 10 es llamada $|V| \cdot \sum_{i=1}^k \sum_{j=i+1}^k 1 = \frac{1}{2} \cdot |V| \cdot (k-1) \cdot k$ y finalmente la asignación de la línea 13 es llamada $|E| \cdot k$ veces. El algoritmo toma entonces un tiempo total $\mathcal{O}(|V|k^2 + |E| \cdot k)$, lo que es tiempo polinomial (para k fijo).
3. Consideramos el siguiente algoritmo:

Algorithm 2: GraphKColoreability(G, k)

Input: Grafo $G = (V, E)$, número de colores k

Output: *true* si G es k -coloreable, *false* en caso contrario

- 1 $\varphi_G \leftarrow \text{GraphKColoringToSAT}(G, k)$
 - 2 **return** SAT-Solver(φ_G)
-

El algoritmo resuelve el problema: G es k -coloreable si y sólo si φ_G es satisfactible, si y sólo si SAT-Solver(φ_G) retorna *true*.

4. El algoritmo usa tiempo $\mathcal{O}(|V|k^2 + |E| \cdot k)$ para construir φ_G , que es de la forma:

$$\varphi_G = \bigwedge_{v \in V} \left(\left(\bigvee_{i=1}^k c_{v,i} \right) \wedge \bigwedge_{i=1}^{k-1} \bigwedge_{j=i+1}^k (\neg c_{v,i} \vee \neg c_{v,j}) \right) \wedge \bigwedge_{(u,v) \in E} \bigwedge_{i=1}^k (\neg c_{u,i} \vee \neg c_{v,i})$$

Dependiendo de cómo se calcule el tamaño de la fórmula será la complejidad final, pero tomando el tamaño con la definición vista en clases, se puede calcular rápidamente la

cantidad de paréntesis, variables y símbolos como:

$$\begin{aligned}
\#var(\varphi_G) &= |V| \cdot \left(k + \frac{k \cdot (k-1)}{2} \cdot 2 \right) + |E| \cdot k \cdot 2 \\
&= |V| \cdot k^2 + 2 \cdot |E| \cdot k \\
\#pars(\varphi_G) &= 2 + \left(2 \cdot (|V| - 1) \cdot \left(2 + 2 \cdot (k-1) + 2 \cdot \sum_{i=1}^{k-2} \sum_{j=i+1}^{k-1} 6 \right) + 2 \cdot (|E| - 1) \cdot (k-1) \cdot 6 \right) \\
&= 2 + (2 \cdot |V| - 2) \cdot \left(2 \cdot k + 2 \cdot 6 \cdot \frac{1}{2} (2 - 3k + k^2) \right) + (2|E| - 2) (6k - 6) \\
&= 2 + (2 \cdot |V| - 2) \cdot (12 - 16 \cdot k + 6 \cdot k^2) + 12 \cdot |E| \cdot k - 12 \cdot |E| - 12k + 12 \\
&= \dots \\
\#symb(\varphi_G) &= 1 + \left((|V| - 1) \cdot \left(1 + (k-1) + \sum_{i=1}^{k-2} \sum_{j=i+1}^{k-1} 3 \right) + (|E| - 1) \cdot (k-1) \cdot 3 \right) \\
&= \dots
\end{aligned}$$

En esta formulación, $n = \#var(\varphi_G) + \#pars(\varphi_G) + \#symb(\varphi_G)$. La expresión resultante en cualquiera de las formas debería ser $n \in \mathcal{O}(|V| \cdot k^2) + |E| \cdot k$, de manera que el algoritmo tendrá tiempo total dado por:

$$\mathcal{O}(|V| \cdot k^2) + |E| \cdot k + f(n)$$

Pauta (6 pts.)

- 1 pt. por una explicación similar, 0 en caso contrario.
- 2 pts. 0.5 pt. de bonus si es que menciona que si k es considerado parte del input, el tiempo no es polinomial, puesto que el tamaño de k es logarítmico respecto a su valor.
- 1 pt. por armar correctamente el algoritmo (o una versión equivalente, me imagino que habrá casos donde descompongan la línea 2 en un if/else con true/false).
- 2 pts. Pueden usarse tamaños alternativos como la cantidad de variables o acotar derechamente la cantidad de paréntesis y símbolos como 3 veces la cantidad de variables para simplificar el razonamiento y se considerará correcto.